Description of the laboratory work No. 1

GENERAL INPUT/OUTPUT OF 32 BIT MICROCONTROLLERS

1. Objectives
2. Preparation for the work
3. Workflow
   3.1. Work with a teacher
      3.1.1. STM32CubeMX
      3.1.2. Template of the code
      3.1.3. Main functions
   3.2. An example of an individual laboratory work assignment
   3.3. Work by individual task
   3.4. Literature

1. Objectives

Get to know the architecture, functionality and configuration features of the general I/O of 32 bit microcontrollers. To develop abilities to analyze microcontrollers specifications. Get to know the software, which is needed for program of 32 bit microcontrollers.

2. Preparation for the work

We will start to work with 32-bit microcontrollers during this laboratory work. Students will try to accomplish several tasks independently after getting to know the basics.

3. Workflow

3.1. Work with a teacher

The lecturer demonstrates boards and explains their components and programming features during the first laboratory work.

3.1.1. STM32CubeMX

STM32CubeMX is a software package that allows you to perform project configuration work in the graphical interface. First, when you run the STM32CubeMX software for the first time, it will be necessary to create a new project

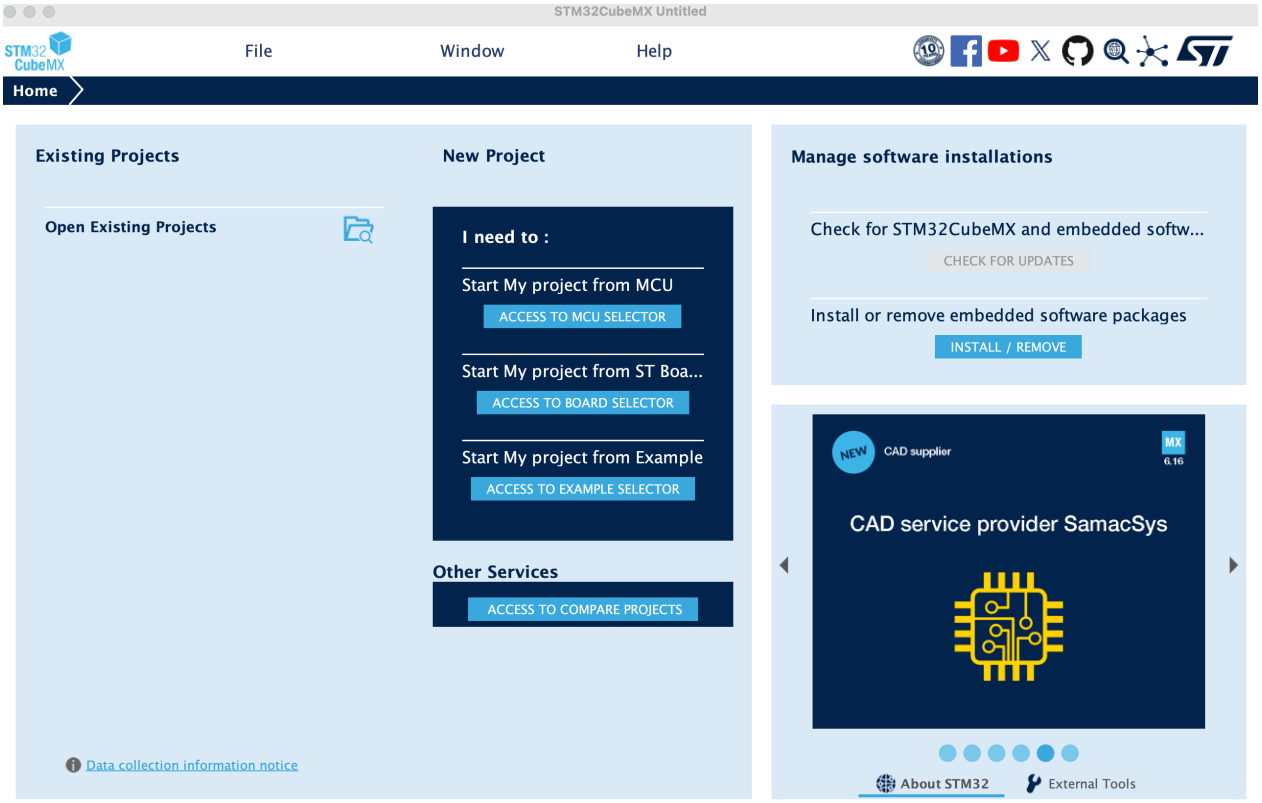


Fig. 1. The dialog window for selection of the home page

NOTE: The STM32CubeMX version may vary by laboratory, so the program window may look slightly different. Just have a look at where to choose the right configuration.

In the main configuration window, it is necessary to press the “Access to board selector” button. As a result, the board selection window will open (Fig. 2).

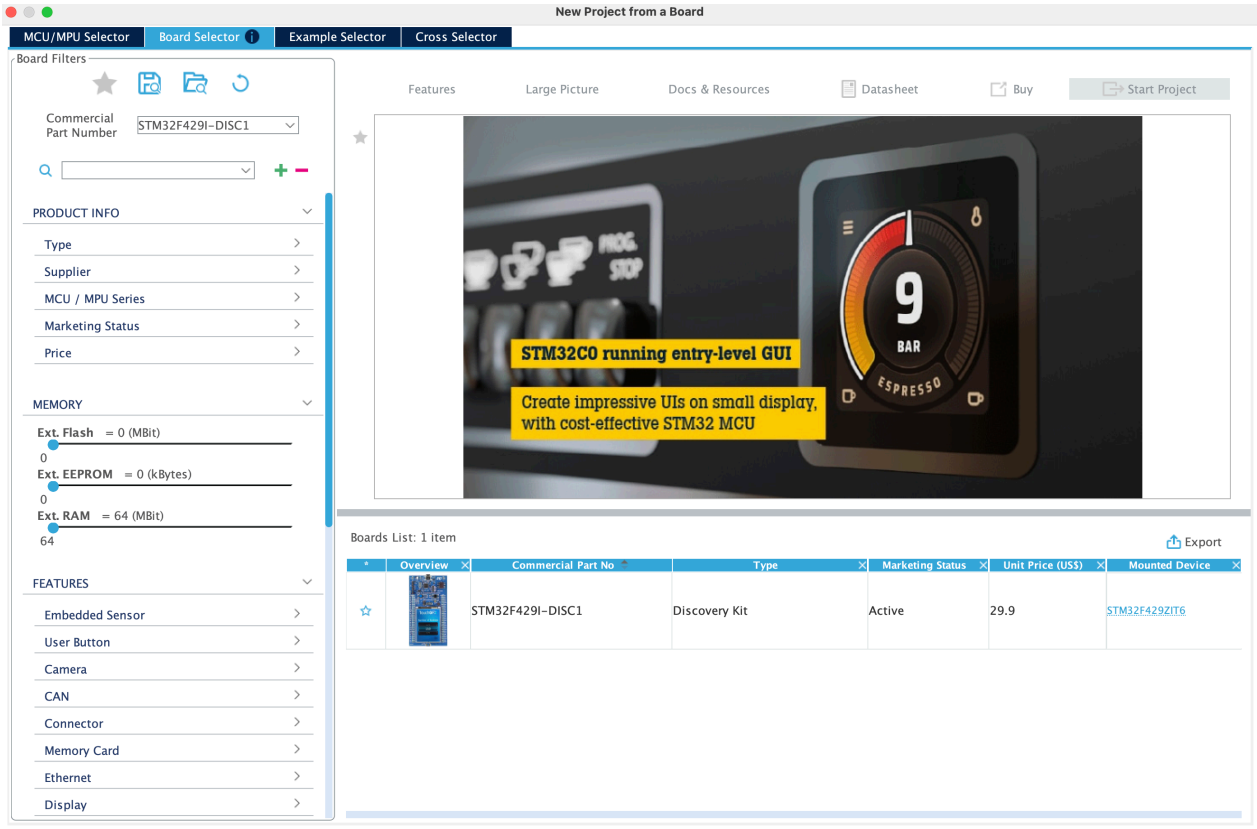


Fig. 2. Board selection window

The following steps in the board selection window should be made (Fig. 2):

1. Start to write “STM32F429I-DISC1” in the search edit box.
2. Select the necessary STM32F429I-DISC1 board from the list.
3. Press “*Start Project*” to be directed to the next configuration window (Fig. 4).

The pinout configuration window will be presented next (Fig. 3).

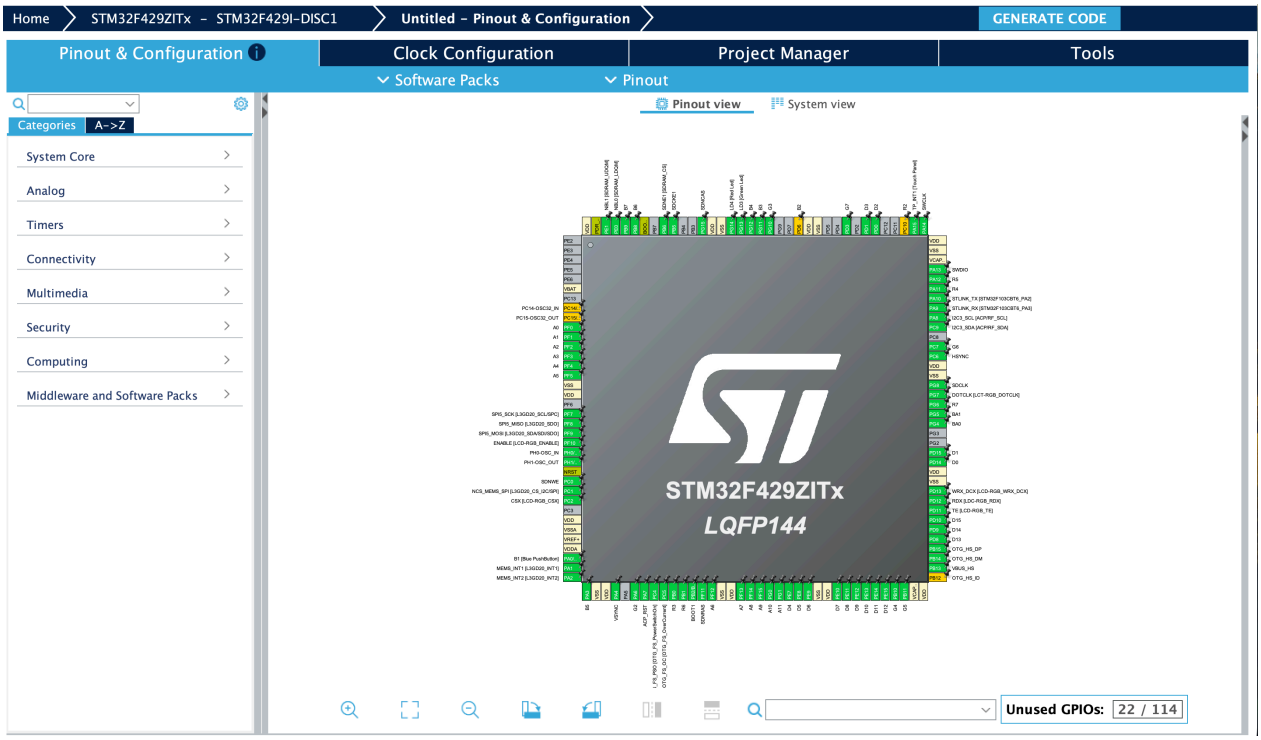


Fig. 3. Project configuration window

First, it is necessary to clear all the configuration of all pins of the microcontroller (Fig. 3): 1 – select *Pinout dropdown* menu; 2 – pick *Clear pinouts*. Then confirm the pinout clear operation (Fig. 4).

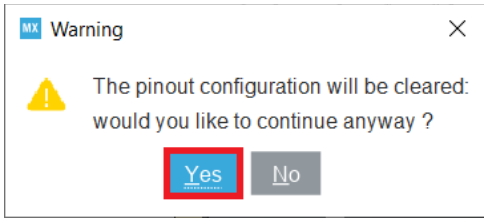


Fig. 4. Confirmation of pinouts clear

The next step is to press the left mouse button on the selected pin and to set the direction of these pins as “GPIO_Output” (Fig. 5). It is also necessary to set the label of every pin: PG13 – green LED; PG14– red LED. It could be done after pressing the right mouse button on the selected pin.

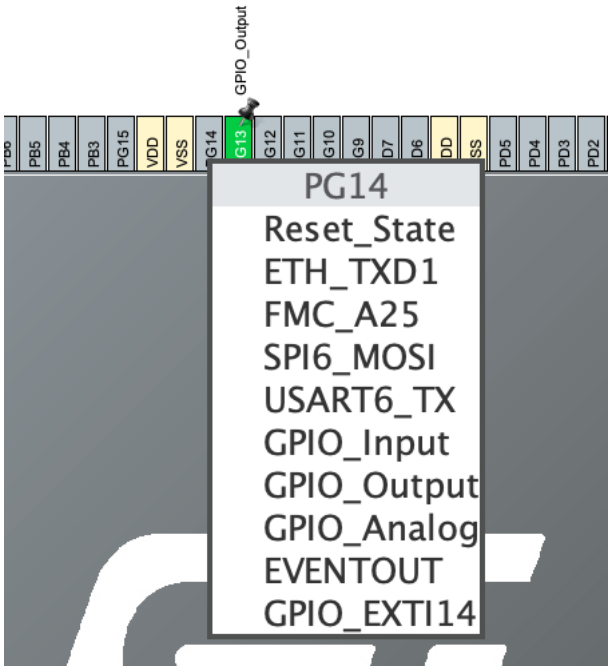


Fig. 5. Configuration of output pins

The configuration of input pin PA0 for the USER BUTTON is made using the same logic (Fig. 6).

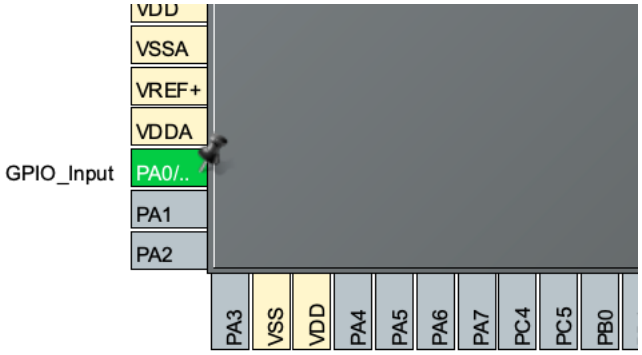


Fig. 6. Configuration of input pin

The next step is to go to the “Clock configuration” tab and to check if there are no errors (Fig. 7). There are usually no problems with the clock source, especially when performing basic operations with standard input output.

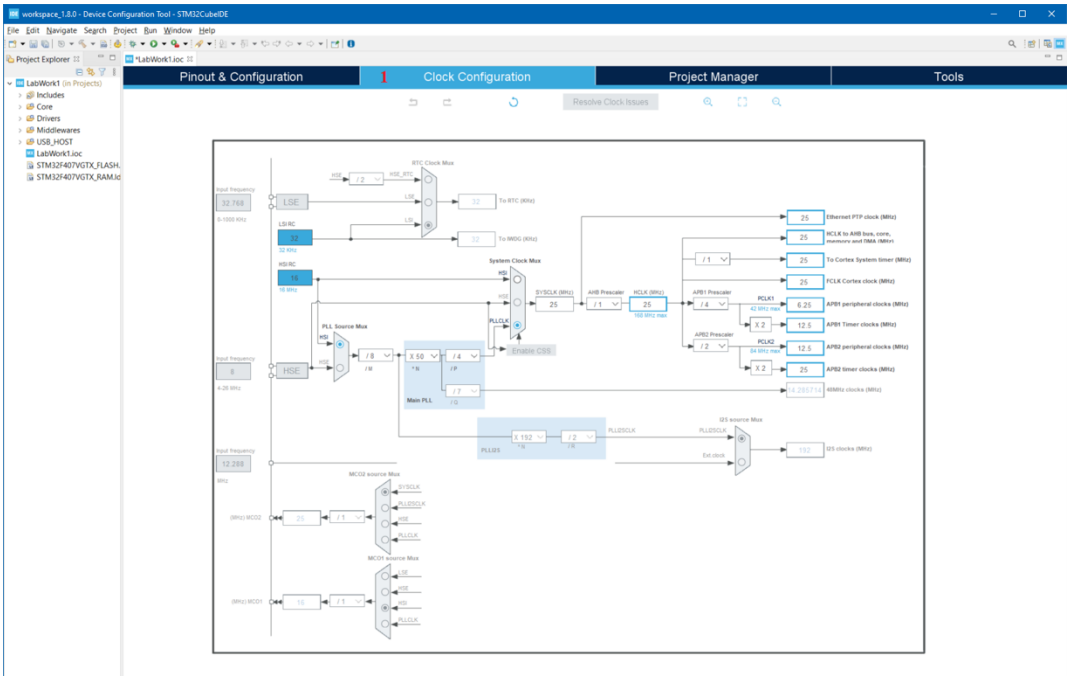


Fig. 7. Clock source configuration window

If there were any problems with the clock source, the red exclamation points would be shown in the clock configuration window and the STM32CubeMX will show the dialog window for automatic configuration of clock source (Figure 8).

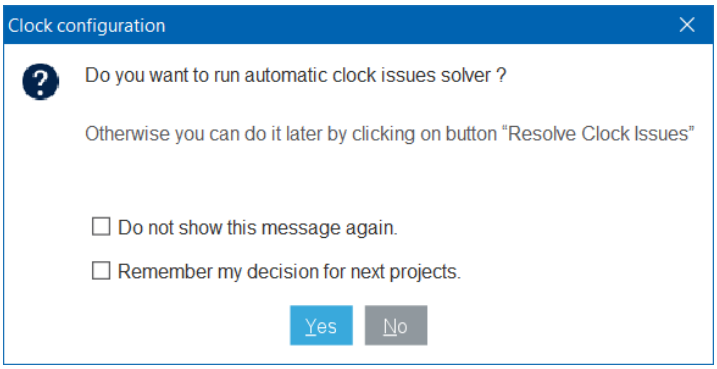


Fig. 8. Dialog window for selection of the automatic clock issues solver.

Automatic clock source matching can also be selected for the first laboratory work. We will talk about the selection of the clock source in detail in later lectures.

Next, we will jump to the "Project Manager" configuration window (Fig. 9).

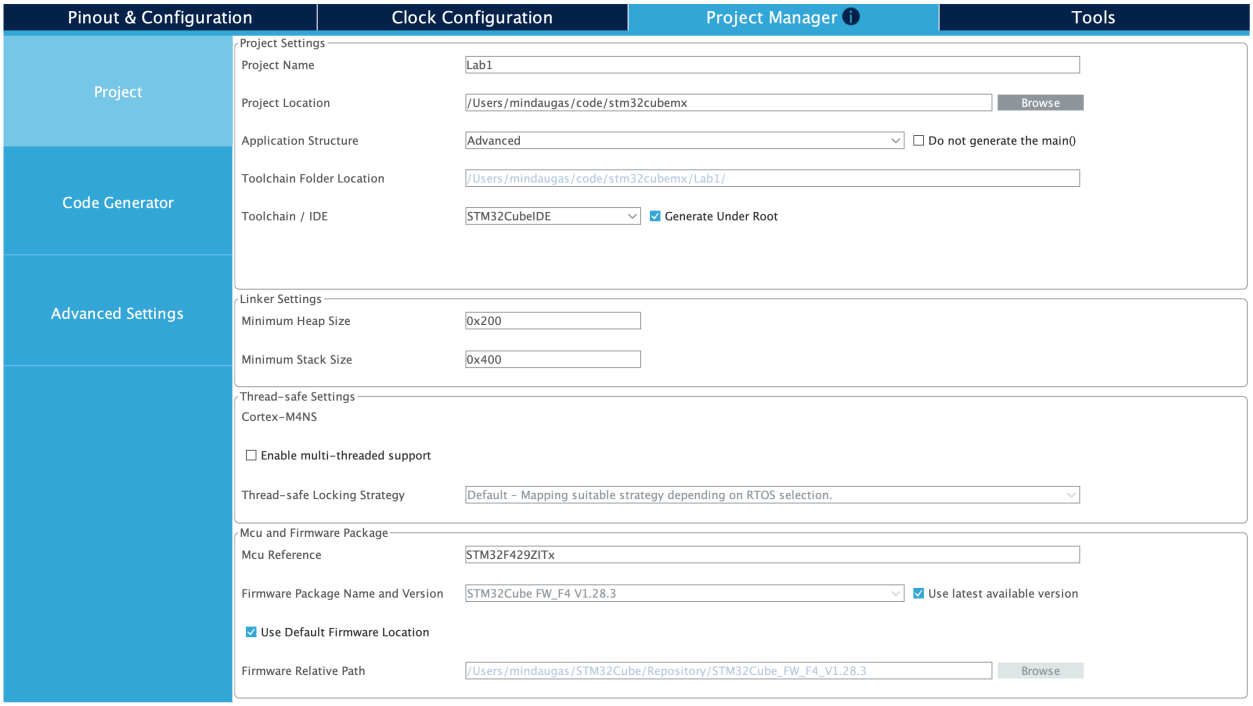


Fig. 9. Project configuration window

Update project settings with your project and location. Next, we click “Generate Code” button in the top right corner (Fig. 10).

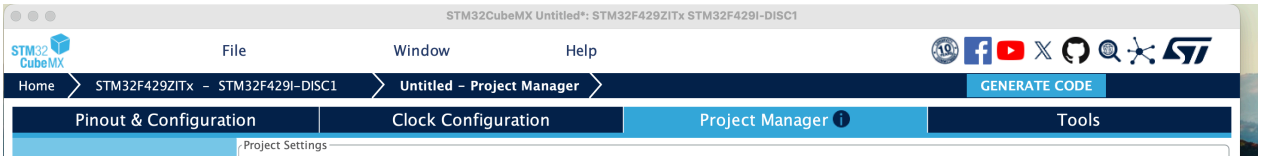


Fig. 10. Generate code

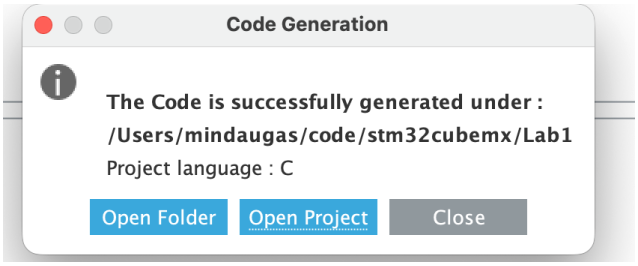


Fig. 11. Code generated

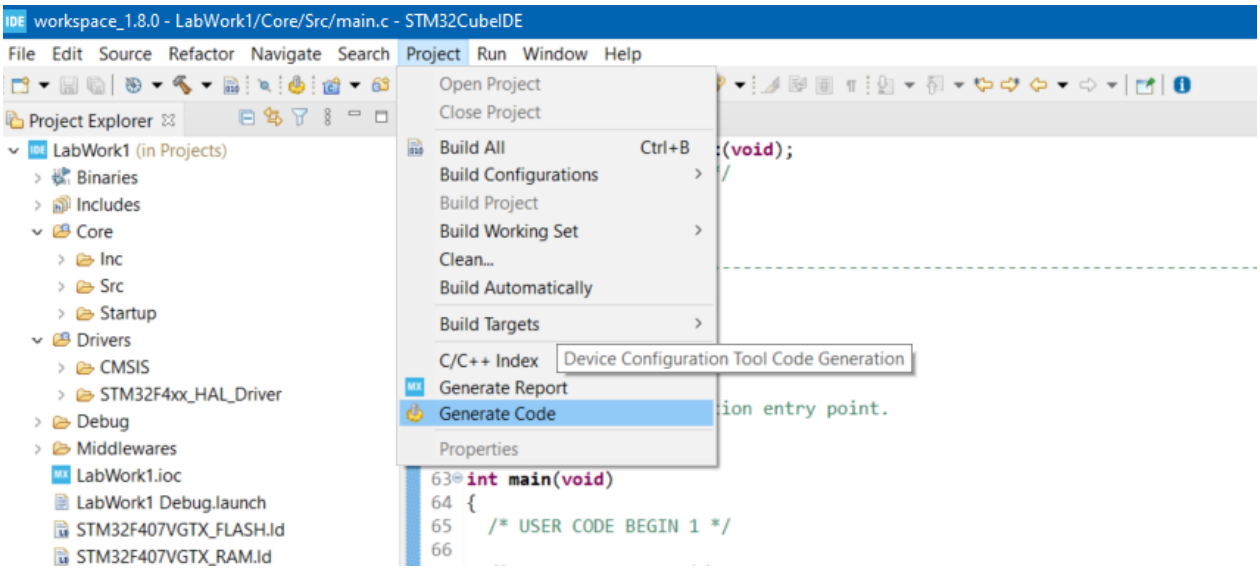


Fig. 12. Steps in order to generate the code files of the project.

The structure of the generated project is presented in (Fig. 13).

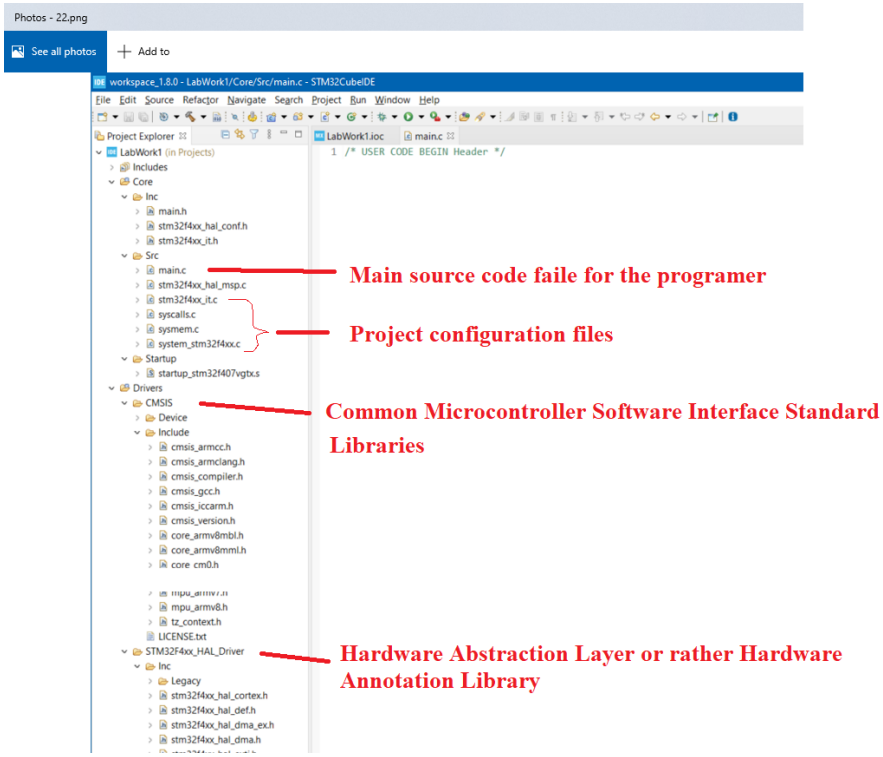


Fig. 13. The structure of generated project

For the programmer, the most important file to edit is the main.c file. Let's open the main.c file and start programming (Fig. 14).

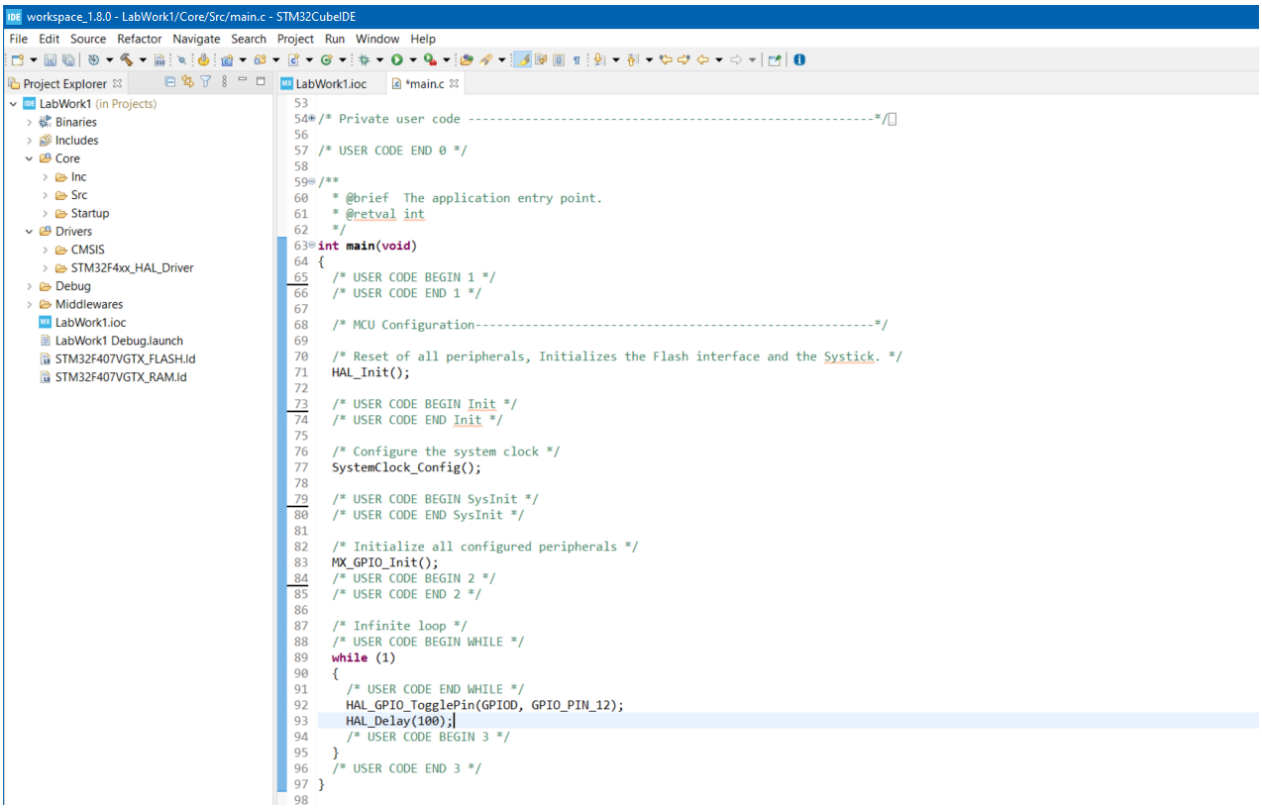


Fig. 14. The structure of the main.c file

The main part of the main.c file is shown in figure 14. This part is the most important for the programmer while doing the task of the first laboratory work. The program is executed from the beginning of the ***main*** function. All other functions should be initialized at the beginning of the main function before the while loop. A while loop is an infinite loop that repeats during every iteration of *CPU*. Currently, two functions are placed in an infinite loop as an example. The first function (line 92) changes the voltage level at pin 12 of terminal D to the opposite. Line 93 provides a delay function that delays *CPU* operation by 100 ms.

Only the standard settings of the STM32CubeIDE will be used during the first laboratory work. It is necessary to compile the project and to send the generated output file to the flash memory of

the microcontroller after modifying the program code. This will require the following steps (Figure 15): 1 - Menu -> Project -> Build Project; 2 - Menu -> Run -> Run.

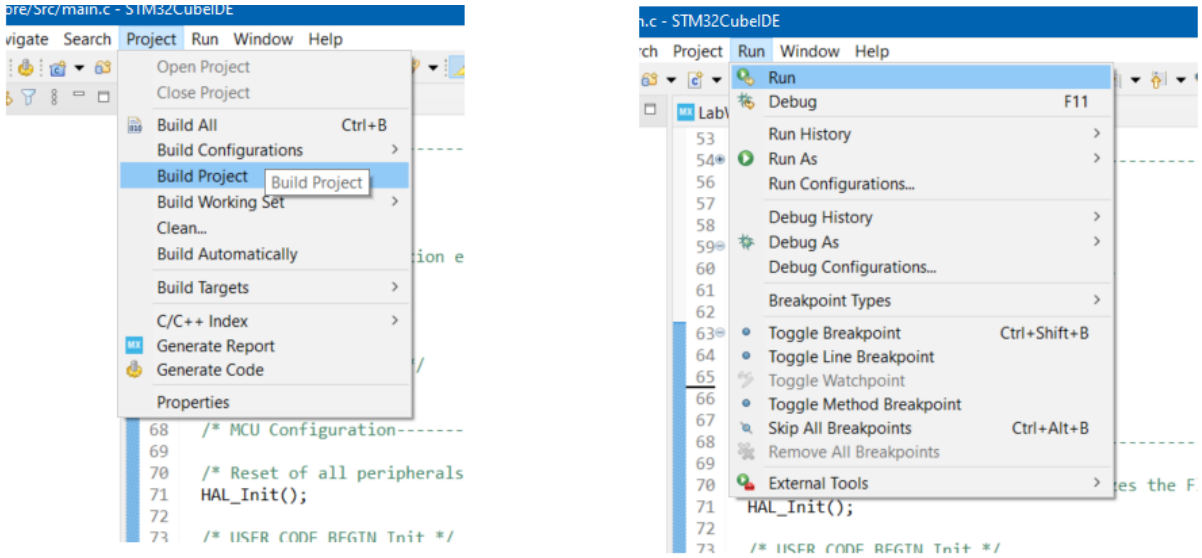


Fig. 15. The sequence of actions in order to compile the project and to upload the output file to the flash memory of the microcontroller

All sorts of errors can occur when compiling a project and sending it to the flash memory of the microcontroller. Errors that occur in all cases will be displayed in the terminal window below.

3.1.2. Template of the code

```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
```

3.1.3. Main functions

// Toggle the voltage level on the pin 13

```
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
```

```
// Set the high voltage level on the pin PD14
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);

// Set the low voltage level on the pin PD13
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_RESET);

// Read the voltage level on the pin PA0
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0

// Delay function. In this particular case the delay of 100 ms
HAL_Delay(100);
```

3.3. Work by individual task

Students independently perform individual work assignments provided by the lecturer. Students can use any additional literature but cannot talk to each other. When faced with difficulties and finding no solution, students can always ask specific questions of the teacher.

3.4. Literature

1. Slides of the lectures in the moodle.vilniustech.lt system.
2. Code examples in the internet.
3. STM32F407. In the moodle system.
4. STM32F407 Discovery. In the moodle system.
5. UM1467 Getting started with software and firmware environments for the STM32F4DISCOVERY Kit. In the moodle system.
6. UM1472 Discovery kit for STM32F4. In the moodle system.
7. DM00031020 stm32f4x reference manual registers level. In the moodle system.